

答辩演讲稿

使用方式

- 对应展示文件: [index.html](#)
 - 主讲时长: 20 分钟
 - 答疑时间: 单独计算
 - 参考节奏:
 - 封面 + 目录: 2 分钟
 - 背景与目标: 3 分钟
 - 工程流程与实现: 5 分钟
 - 关键代码与原理: 4 分钟
 - 难点与治理: 3 分钟
 - 落地价值与材料索引: 3 分钟
-

第 1 页: 封面

老师们好, 我的课题是“基于大数据开发套件智能体构建全方位评测体系”。其中我选择的子课题是发布 Agent 方向里的 Hive SQL 评测集沉淀与 Hive 解析能力验证。

这个项目的重点, 是把评测数据、评测执行、结果分析和后续维护做成一套完整工程, 最终沉淀成可以长期复用的评测底座。

页面右侧的几个数字可以直观展示这个项目的规模。当前原始数据抓取总量是 4061 条, 去重后高可信候选池是 2457 条, 最终沉淀为 100 条最终黄金集。同时, 这 100 条并不是静态表格, 而是把来源侧真值、历史对照结果和本次运行结果一起沉淀下来, 再配上评测报告和版本治理机制, 形成一整套可维护的数据资产。

第 2 页: 目录

接下来我的汇报将分为以下五个部分。

第一部分是背景与目标, 主要说明课题要求是什么, 以及项目最终交付了什么。第二部分是工程流程与实现, 也就是数据怎么获取、怎么挑选、怎么评测。第三部分是关键代码与原理, 重点讲主链路、评分逻辑和 SDK 桥接。第四部分是难点与治理, 主要是数据的多源真实性、结果字段分层和版本稳定性。最后一部分是应用价值和备查材料, 方便把这套工程放到真实落地场景下去看。

第 3 页: 01 背景与目标-项目背景

这个项目的背景, 直接来自于课题文档对发布 Agent 子方向的要求。课题里强调的不是单点验证, 而是围绕评测数据集搭建和智能响应结果校验, 构建一个自动化、可量化、可维护的完整闭环。

放到 Hive SQL 这个任务上, 核心问题可以归纳成四个。首先是数据从哪里来。第二点是真值标签怎么保证可信。第三点是结果怎么分层记录, 也就是标准答案、历史对照和本次运行结果分别怎么存储。最后是数据集怎么长期维护, 避免每次重跑都换题。

所以这个项目在设计上采用了四个原则。第一, GT 也就是标准答案, 来自于真实的来源断言。第二, baseline 作为原始数据经过 SDK 验证后, 只保留最早冻结下来的历史对照。第三, actual 才代表每次评测实时运行出来的结果。第四, 版本快照和待审池用于支持后续可持续迭代。页面下方这条流程线, 就是从课题要求一步步推导到项目策略、评测策略和治理策略。

在课题文档中, [课题说明原文](#), 有两点可以对照。第一, 课题原文明确要求数据层、评测引擎层和结果输出层分层建设, 所以当前工程结构是和课题拆解直接对应的。第二, 课题强调的是自动化评测研判和完整专业报告, 因此这里交付的不只是数据集, 还包括评分标准、样本约束说明和迭代方案这些配套产物。

第 4 页: 01 背景与目标-交付结果

关于交付结果, 这里重点看四层内容。最核心的是 100 条最终黄金集样本; 第二层是 455 条待审池, 用来隔离中低可信样本, 避免它们直接进入主评测; 第三层是报告体系, 包括主评测报告、评分原理、样本约束说明和黄金集迭代方案; 第四层是代码与版本快照, 用来支撑后续复用和维护。

项目的主要结果是生成了 100% 都是高可信真值样本的最终黄金集, 说明项目在可信度控制上收得很严。

结果中主要有 5 个指标。第一个是解析通过率 48%, 表示 100 条样本里有 48 条被当前 SDK 判成了通过, 因为当前黄金集中实际上只有 51 条正样本, 所以这个数值本身并不低。第二个是正样本通过率 94.12%, 也就是 51 条本来应该通过的样本里, 有 48 条通过验证, 这个指标直接反映正例解析能力。第三个是负样本命中率 100%, 也就是 49 条本来应该失败的样本全部都被判成了失败, 说明负例拦截是到位的。第四个是与标准答案完全一致的比例, 解析文档里把它记作 GT 严格一致率, 当前已经提升到 94%, 其中, 项目把 SDK 原始报错字段和评测对齐字段拆开记录, 绝大多数负样本可以进入到严格一致统计。第五个是达到可用结果的比例, 项目文档里把它记作 GT 可用匹配率, 当前是 97%, 统计的是在 0、1、2 分三类样本中 1 分及以上的结果比例。这 5 个指标体现了当前黄金集的质量是比较可观的。

最终黄金集页面继续看字段设计。*gt_** 这一组字段记录标准答案, *baseline_** 这一组字段记录更早冻结下来的历史对照, *sql_text* 是原始样本, *source_ref* 用来追溯来源。这说明项目不是只有一个结果列, 而是把标准答案、对照信息和来源一起沉淀下来。

待审池页面也能直接说明一点: 待审池不是被淘汰的数据, 而是未来升标和覆盖补洞的重要来源。

主评测报告页面这里还可以看到更细的结果输出。当前报告中有 94 条完全一致、3 条部分匹配、3 条关键判断错误。评分标准页面会再解释, 这三类在项目里分别对应 2 分、1 分和 0 分。这里先强调结果含义, 也就是大部分样本已经实现了严格对齐, 剩余差异只集中在极少数数据库标识符类负样本和 3 条真正样本缺口上。

黄金集迭代页面这里也可以顺势过渡到后面的版本治理, 说明这 100 条并不是随手挑出来的, 而是经过历史快照、稳定核心和待审池机制共同治理的。

第 5 页: 02 工程流程与实现-整体架构

接下来进入工程流程, 也就是这套结果到底是怎么跑出来的。

页面最上方这条流程, 给出了最简洁的主链路: 先从官方文档、开源 benchmark 和 parser unit 三个来源抽取真实样本; 然后做清洗、高可信候选池过滤和去重; 再形成最终 100 条黄金集; 最后执行评测、生成报告并落版本快照。

左下角这张表展示的是代码工程分层。cli 负责一键入口, pipelines 负责数据采集和建集, adapters 负责 Python 到 Java Hive Parse SDK 的桥接, reporting 负责评分与报告输出, core 负责分类规则和通用配置。这样的拆分让项目从几个分散脚本变成了可以重复运行的工程系统。

下方这张完整项目运行流程图, 可以按 5 个步骤理解。第一步是命令入口; 第二步是多源采集与建集; 第三步是在候选池阶段先把历史对照结果, 也就是 baseline, 冻结下来, 并检查最终黄金集样本约束; 第四步是批量解析生成本次运行结果, 也就是 actual 指标结果; 第五步是评分、写出 CSV、Markdown 等报告。整个过程是从数据集建设一直闭环到结果输出的。

run_eval.py 这个页面直接就能说明: 这是完整运行入口, 主函数做的事情很清楚, 先建集, 再加载黄金集执行评测, 再写出结果、能力分析、评分标准、样本约束说明和迭代报告, 这说明项目具备一键跑完整链路的工程能力。

dataset_pipeline.py 这个页面这里重点看两类逻辑: 一类是历史偏好与多样性排序, 说明稳定核心和上一版样本如何影响排序; 另一类是 *select_target_rows*, 说明最终 100 条是如何按四类结构、再平衡和补洞规则选出来的。

第 6 页: 02 工程流程与实现-数据集构建

这一页继续展开最核心的数据集构建逻辑。

先看数量口径。原始抓取总量 4061 条, 是三个来源直接抽取出来的总量。之后清洗得到 3943 条真实候选, 这一步只做有效性过滤, 不做跨来源去重。再往后, 只有高可信真值样本能进入高可信候选池, 所以得到 3488 条。然后按标准化 SQL 去重, 得到 2457 条去重后的高可信候选池样本, 最后从这里面选出 100 条最终黄金集。

这个数量关系里有两个点值得注意。第一点是清洗和去重承担了两类不同工作。清洗完成了删除空 SQL、字段缺失、来源不清楚这类无效样本；去重完成了在高可信候选池里，把标准化后内容相同的 SQL 合并掉。所以项目才会先清洗到 3943 条，再去重到 2457 条。第二点是待审池不是噪声回收站，而是可信度不足但仍然有价值的样本集合，后续阶段中可以通过人工校对更新黄金集并进行版本迭代。

右侧的流程框说明了最终 100 条黄金集数据，具体是怎么形成的。先按 QUERY、DDL、DML、UTILITY 四类结构初选，保证结构分布达标；然后再让正负样本比例接近平衡；再做错误类型补洞，保证负样本粒度覆盖；最后做特性覆盖补洞，补齐 update、delete 等关键语法特性。这个过程不是随机抽样，而是固定排序和覆盖规则共同决定的，所以只要输入候选池不变，最终黄金集通常会收敛到同一版结果。

页面下方是原始数据的三个来源链接。

在 Hive 官方文档页面，这类样本证据最强，特别适合作为强正例来源。

Apache Hive benchmark 来源的价值在于规模和覆盖面，但不同目录的断言强度不完全一样，所以需要来做来源分层，而不是整批照搬。

关于 parser unit 来源，项目不是直接读取 SQL 文件，而是从 Java 单测中恢复 SQL 字符串，用来补强复杂语法和真实负样本。

第 7 页：03 关键代码与原理-评测原理

有了数据集之后，接下来重点解释评测为什么可信。

这一页最重要的就是左上角这条链路，也就是 GT、baseline、actual 这三层关系。GT 对应来源侧给出的标准答案；baseline 对应最早冻结下来的历史对照；actual 对应实时运行返回的结果。项目把这三层显式分开存储和比较，目的是让标准答案、历史参考和当前表现不要混在一起，同时方便后续版本对照和参考。

另外，baseline 和 actual 现在都同时保留两套错误字段：一套是对齐 GT 口径后的评测字段，用来进行打分；另一套是 SDK 原始归一化字段，用来排查解析器真实报错。这样既能保证评测口径稳定，又不会把 SDK 原始信息丢掉。

在评分上，项目也没有只停留在 pass/fail，而是用了三档结果。这里还是 GT，作为参照。完全一致记为 2 分，部分匹配记为 1 分，关键判断错误记为 0 分。这样的设计比单一通过率更有解释性，也更适合定位问题。

右侧这张表展示了项目的主要评测结果。

在评分标准页面中，这里重点讲评分图在当前项目中的适配。因为当前每个 case 都是单条 SQL，没有真实表结构、权限和运行上下文，所以这套评测只能严肃验证语法解析能力，不能把执行语义混进来。也就是说，这里的 GT 不是语义执行结果，而是这条 SQL 在 Hive 语法层面应当 pass 还是 fail，以及失败时的错误类型和子类型。

在样本约束说明页面中，主要有两个重点。第一，最终黄金集只保留高可信真值样本，所以主评测集本身是收严过的。第二，baseline 在候选池阶段冻结，actual 在评测阶段实时生成，它们承担的是历史对照和本次结果这两类不同职责。

在主评测报告页面中，报告页里除了总体指标，还给出了三档评分分布、一级分类严格一致率和二级类型覆盖，这些内容更适合用来定位解析器当前的强项和短板。当前严格一致率已经达到 94%，其中 49 条负样本里有 46 条达到了严格一致，剩余 3 条 partial 都集中在数据库标识符类 case。

第 8 页：03 关键代码与原理-关键代码

接下来是项目的关键代码。

这一页主要沿着主链路讲四件事。第一，入口脚本如何把全流程串起来；第二，选择逻辑如何保证结构和覆盖；第三，评分逻辑为什么能得到三档结果；第四，SDK 桥接如何让 Python 工程和 Java 解析能力结合在一起。

先看左上角的代码片段 run_eval.py。这段代码的作用是把建集、评测、写结果、写报告这些动作统一在主函数里执行，所以项目具备一键运行能力。

再看右上角的选择逻辑。项目按四类结构取样，再做正负平衡、错误类型补洞和特性补洞。

左下角的评测部分 `evaluator` 负责把标准答案和当前结果逐项比对，除了 `pass` 和 `fail`，还包括比较错误类型和错误子类型，所以最后形成完全一致、部分匹配、关键错误这三档结果。

右下角的 SDK 桥接体现的是实现思路：主流程保持 Python，利于开发和维护；真正的解析能力由 Java Hive Parse SDK 提供，这样可以兼顾工程效率和底层一致性。

`run_eval` 页面一句话概括就是：这是“建集 -> 评测 -> 报告”的完整入口。

`dataset_pipeline` 页面重点就讲历史偏好排序和 `select_target_rows`，说明稳定核心、上一版样本保护和最终选样规则都在这里落地。

`evaluator` 页面要讲的重点是，这段逻辑不只判断 `pass/fail`，而是把状态和错误粒度一起纳入评分；同时 `baseline` 和 `actual` 都会同时保留对齐字段与 `raw` 字段，因此报告才能既可评分、又可排查。

SDK 桥接页面补充说明的是，这一层负责编译 `Java CLI`、组织输入输出、批量调用 SDK，是整个工程能接入 `Hive` 原生 `ParseDriver` 的关键适配层。

第 9 页：04 难点与治理-问题与解法

接下来是项目里的难点和治理。

第一个难点是数据多源真实性差异。官方文档、benchmark 和 parser unit 的证据强度并不相同，所以项目没有简单混用，而是做了高可信、中等可信、低可信分层，最终黄金集只保留高可信真值样本，中低可信样本统一进入待审池。

第二个难点是版本抖动，也就是固定结构下的近似样本在不同版本之间可能发生小范围轮换。这里的处理方式是优先保留稳定核心和上一版样本，在仍然允许再平衡和覆盖补洞的前提下，减少无意义漂移。

第 10 页：04 难点与治理-版本治理

关于版本治理，当前项目因为候选池、待审池和约束条件都不变，所以选样逻辑通常会收敛到同一个结果。

这里有三个概念需要区分开来。首先是初始黄金候选集是第一版可用结果，其次最终黄金集是结合历史快照和稳定核心策略后收敛出来的版本，而待审池则是后续版本演进的重要输入。

页面下方这条流程线展现了版本演进如何发生：当来源刷新、人工升标或覆盖缺口变化导致候选池变化时，系统会先读取历史快照中的稳定核心，再优先保留核心，只在确实有净收益时形成下一版黄金集。

在版本索引页面中，文档记录了最新版本号、快照路径和更新时间，便于后续追踪当前黄金集来自哪一次版本沉淀。

第 11 页：04 难点与治理-3 个 0 分样本补充页

这一页补充了当前 3 个 0 分样本的情况。

这 3 条样本分别是 `HIVE_REAL_016`、38 和 40，前两条类别属于 `WINDOW`，后一条属于 `LATERAL_VIEW`。它们的共同点是标准答案都来自 Hive 官方文档，都是高可信正样本，但当前本地 SDK 在历史对照结果和本次运行结果里都返回了 `fail`，所以最终得到 0 分。

这里最关键的结论有两点。第一，这 3 条样本不是版本轮换边缘引入的问题样本，因为它们在冻结下来的 `baseline` 阶段就已经验证失败了，并不是这次 `actual` 才新出现的退化。第二，它们之所以仍然保留在黄金集里，是因为这几条样本同时承担着官方文档高可信和关键特性覆盖的作用。

也就是说，这 3 条样本满足“覆盖缺口必须保留”，但不满足“当前解析器通过”。所以当前项目在这里的取舍，是保留真实能力缺口，而不是通过替换样本抬高通过率。

第 12 页: 05 落地价值与材料-应用价值

最后从应用落地角度看这个项目。

第一是效率价值。项目把原来分散、人工化的 SQL 验证工作，整理成了自动化评测流水线。第二是复用价值。数据集、评分逻辑和报告链路是解耦的，如果后续接入新的 Parser 或 Agent，这套底座仍然可以复用。第三是治理价值。因为有待审池、版本快照和样本约束说明，数据集不再是一次性表格，而是可以长期运营的资产。

同时，这个项目还有很多可以优化的地方。比如当前项目主要验证语法解析能力，还没有进入真实执行语义层；待审池升标仍需要人工参与；完整重跑还有缓存优化空间；部分特性覆盖仍可以继续补强。

总结来说，当前项目最大的价值，是沉淀了一套可以继续扩充、继续复用、也可以继续治理的评测工程。

第 13 页: 05 落地价值与材料-材料索引

这一页主要作为答辩过程中的文件导航。

关于数据真实性，可以从左侧的最终黄金集、待审池和三个来源页面观察；关于评测逻辑，可以从右侧的主评测报告、评分原理和样本约束说明文档观察；关于代码实现，可以继续从运行入口、建集总控、评分逻辑和 SDK 桥接这几个文件观察。

第 14 页: 感谢聆听-Q&A

以上就是汇报的主要内容，感谢各位老师聆听。

备用讲法

1. 如果老师一开始就追问“这个项目和普通脚本有什么区别”

可以直接回答：普通脚本更像一次性验证，而当前项目同时解决了数据来源、真值分层、结果字段分层、报告输出和版本治理五件事，所以它更接近一套完整评测工程。

2. 如果老师追问“为什么不是直接看解析通过率”

可以回答：因为当前黄金集中本身包含负样本，负样本应该失败，所以通过率不能单独代表能力。更合理的看法是把正样本通过率、负样本命中率、和标准答案完全一致的比例，以及三档评分分布一起看。

3. 如果老师追问“为什么一定要保留待审池”

可以回答：待审池的作用不是存放废数据，而是隔离中低可信样本，保证主评测集只保留高可信真值样本，同时为后续升标和覆盖补洞保留来源。

4. 如果老师追问“为什么版本治理还有必要”

可以回答：固定输入下确实不需要重复跑很多次，但当候选池刷新、待审池升标或覆盖约束变化时，版本治理可以保证系统优先保留稳定核心，只做有净收益的替换。

5. 如果老师追问“当前项目最核心的技术点是什么”

可以回答：最核心的不是单个函数，而是三件事的组合落地：真实来源给出的标准答案、历史对照和本次结果的分层记录，以及带有稳定核心约束的黄金集迭代机制。